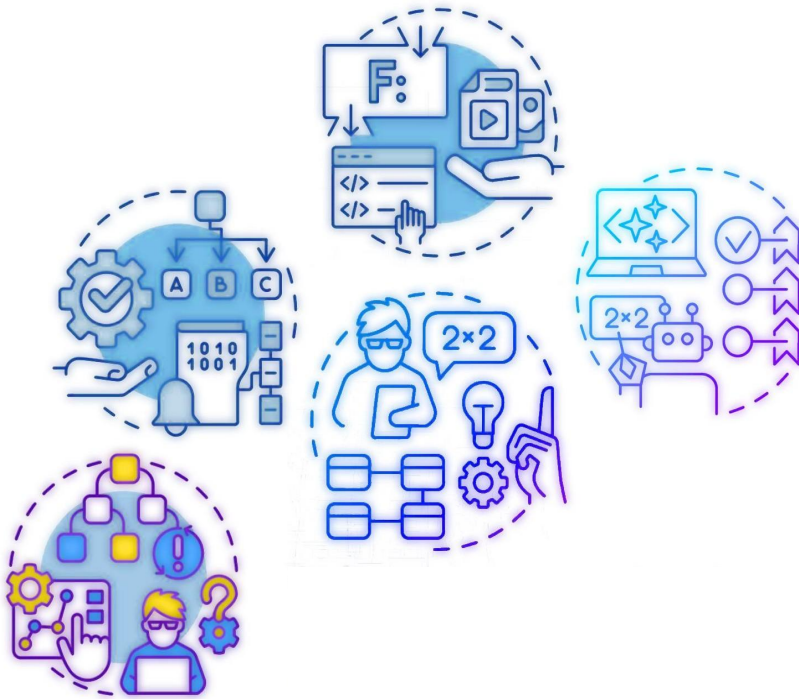




Paradigma Orientado a Objetos

Esp. Ing. Viviana A. Santucci
Ing. J. Exequiel Benavidez
Ing. Jimena Bourlot
Dr. Ing. O. Cristian Martínez

Tecnologías de la Programación

Paradigma Orientado a Objetos

Programa



Conjunto de objetos
que colaboran entre sí
enviándose mensajes

Envío de mensajes

```
1 class Persona:
2     def __init__(self, nombre, edad):
3         self.nombre = nombre
4         self.edad = edad
5
6     def saludar(self):
7         return f"Hola, soy {self.nombre} y tengo {self.edad} años."
8
9 p = Persona("Juan", 30)
10 print(p.saludar()) # Salida: Hola, soy Juan y tengo 30 años.
```

p envía un mensaje al método saludar().

Abstracción

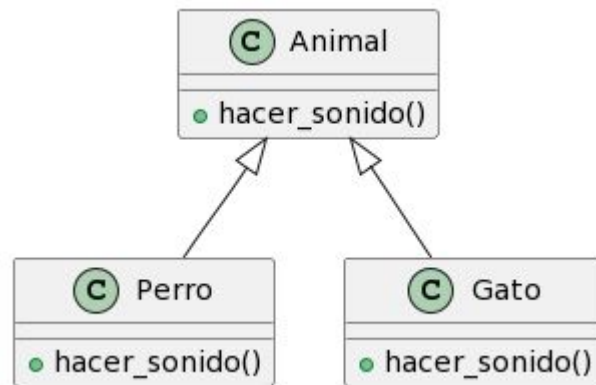
Representar entidades del mundo real con características (**atributos**) y comportamientos (**métodos**).

```
1 class Auto:
2     def __init__(self, marca, modelo):
3         self.marca = marca
4         self.modelo = modelo
5
6     def mostrar_info(self):
7         return f"Auto {self.marca}, Modelo {self.modelo}"
8
9 a = Auto("Toyota", "Corolla")
10 print(a.mostrar_info()) # Salida: Auto Toyota, Modelo Corolla
```

Herencia

Permite definir una clase a partir de otra ya existente, compartiendo estructura y comportamiento.

```
1 class Animal:
2     def hacer_sonido(self):
3         pass
4
5 class Perro(Animal):
6     def hacer_sonido(self):
7         return "Guau!"
8
9 class Gato(Animal):
10    def hacer_sonido(self):
11        return "¡Miau!"
12
13 perro = Perro()
14 gato = Gato()
15 print(perro.hacer_sonido()) # Salida: Guau!
16 print(gato.hacer_sonido()) # Salida: ¡Miau!
```



Encapsulamiento

```
1 class CuentaBancaria:
2     def __init__(self, saldo):
3         self.__saldo = saldo # Atributo privado
4
5     def depositar(self, cantidad):
6         self.__saldo += cantidad
7
8     def obtener_saldo(self):
9         return self.__saldo
10
11 cuenta = CuentaBancaria(1000)
12 cuenta.depositar(500)
13 print(cuenta.obtener_saldo()) # Salida: 1500
```

Polimorfismo

Invocar métodos distintos con el mismo nombre en distintos objetos

```
1 class Figura:
2     def area(self):
3         pass
4
5 class Rectangulo(Figura):
6     def __init__(self, base, altura):
7         self.base = base
8         self.altura = altura
9
10    def area(self):
11        return self.base * self.altura
12
13 class Circulo(Figura):
14     def __init__(self, radio):
15         self.radio = radio
16
17     def area(self):
18         return 3.14 * self.radio ** 2
19
20 figuras = [Rectangulo(5, 10), Circulo(3)]
21 for figura in figuras:
22     print(figura.area())
```


Interfaces

Establecen un contrato de métodos que deben ser implementados por las clases hijas

```
1 // Definición de la interfaz en Java
2 interface Animal {
3     void hacerSonido();
4 }
5
6 // Clase que implementa la interfaz en Java
7 class Perro implements Animal {
8     @Override
9     public void hacerSonido() {
10         System.out.println("Guau!");
11     }
12 }
13
14 // Clase que utiliza la interfaz en Java
15 public class Main {
16     public static void main(String[] args) {
17         Animal animal = new Perro();
18         animal.hacerSonido(); // Salida: Guau!
19     }
20 }
```

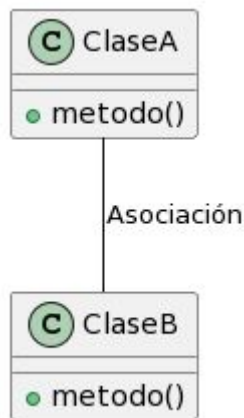


```
1 # Importar ABC (Abstract Base Classes) del módulo abc
2 from abc import ABC, abstractmethod
3
4 # Definición de la interfaz en Python
5 class Animal(ABC):
6     @abstractmethod
7     def hacer_sonido(self):
8         pass
9
10 # Clase que implementa la interfaz en Python
11 class Perro(Animal):
12     def hacer_sonido(self):
13         print("Guau!")
14
15 # Clase que utiliza la interfaz en Python
16 if __name__ == "__main__":
17     animal = Perro()
18     animal.hacer_sonido() # Salida: Guau!
```



Asociación

Una clase "usa" a la otra

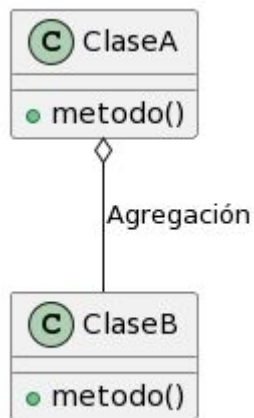


```
1 class Empresa:
2     def __init__(self, nombre):
3         self.nombre = nombre
4
5 class Empleado:
6     def __init__(self, nombre, empresa):
7         self.nombre = nombre
8         self.empresa = empresa
9
10 empresa = Empresa("Arcor")
11 empleado = Empleado("Juan", empresa)
```

An orange arrow points from the `empresa` argument in the `Empleado` constructor call on line 11 to the `Empleado` class definition on line 5.

Agregación

Una clase es "parte de" otra clase

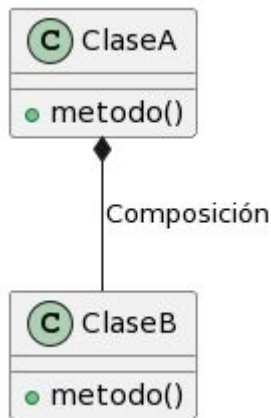


```
1 class Motor:
2     def __init__(self, tipo):
3         self.tipo = tipo
4
5 class Coche:
6     def __init__(self, marca, motor):
7         self.marca = marca
8         self.motor = motor
9
10 motor_coche = Motor("Gasolina")
11 coche = Coche("Toyota", motor_coche)
```

Composición

Aún más fuerte que la agregación.

Si la clase contenedora se destruye, también lo hace la clase contenida.

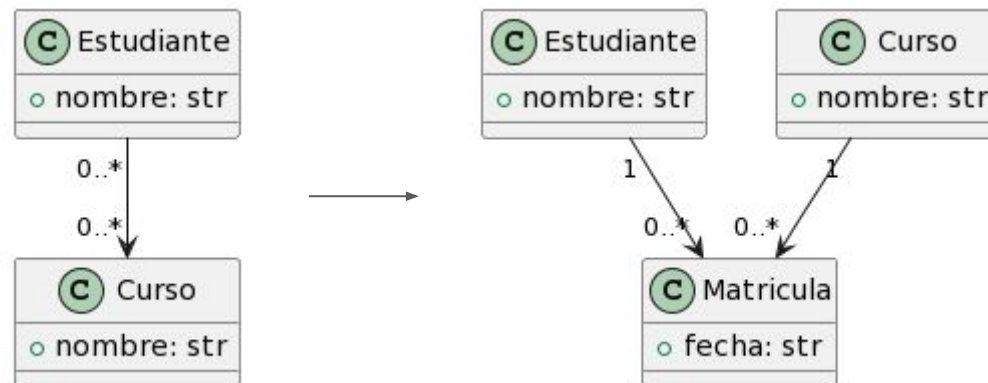


```
1 class Cerebro:
2     def __init__(self):
3         self.pensamientos = []
4
5     def agregar_pensamiento(self, pensamiento):
6         self.pensamientos.append(pensamiento)
7
8 class Persona:
9     def __init__(self, nombre):
10        self.nombre = nombre
11        self.cerebro = Cerebro()
12
13 persona = Persona("Alice")
14 persona.cerebro.agregar_pensamiento("¡Hola mundo!")
```

Composición

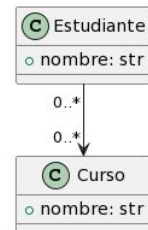
1..*

Clase asociación



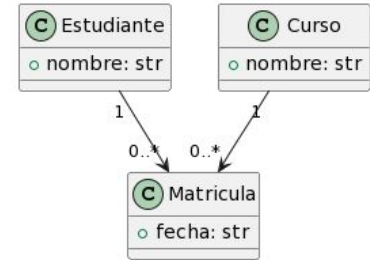
Clase asociación

```
1 class Estudiante:
2     def __init__(self, nombre):
3         self.nombre = nombre
4         self.cursos = []
5
6     def inscribir_curso(self, curso):
7         self.cursos.append(curso)
8
9 class Curso:
10     def __init__(self, nombre):
11         self.nombre = nombre
12         self.estudiantes = []
13
14     def agregar_estudiante(self, estudiante):
15         self.estudiantes.append(estudiante)
16
17 # Crear instancias de Estudiante y Curso
18 estudiante1 = Estudiante("Juan")
19 estudiante2 = Estudiante("María")
20 curso1 = Curso("Matemáticas")
21 curso2 = Curso("Historia")
22
23 # Inscribir estudiantes en cursos
24 estudiante1.inscribir_curso(curso1)
25 estudiante1.inscribir_curso(curso2)
26 estudiante2.inscribir_curso(curso1)
27
28 # Agregar estudiantes a cursos
29 curso1.agregar_estudiante(estudiante1)
30 curso1.agregar_estudiante(estudiante2)
31 curso2.agregar_estudiante(estudiante1)
```



Clase asociación

```
1 class Estudiante:
2     def __init__(self, nombre):
3         self.nombre = nombre
4
5 class Curso:
6     def __init__(self, nombre):
7         self.nombre = nombre
8
9 class Matricula:
10     def __init__(self, estudiante, curso, fecha):
11         self.estudiante = estudiante
12         self.curso = curso
13         self.fecha = fecha
14
15 # Crear instancias de Estudiante y Curso
16 estudiante1 = Estudiante("Juan")
17 estudiante2 = Estudiante("María")
18 curso1 = Curso("Matemáticas")
19 curso2 = Curso("Historia")
20
21 # Crear instancias de Matricula
22 matricula1 = Matricula(estudiante1, curso1, "2024-01-01")
23 matricula2 = Matricula(estudiante1, curso2, "2024-01-15")
24 matricula3 = Matricula(estudiante2, curso1, "2024-02-01")
```



Normas de Estilo para programar en Python



**Es requisito que
se cumplan en el
TP**

PEP 8 → *cómo escribir código **legible***

Define convenciones de nombres, formatos

PEP 484 → *cómo anotar **tipos***

Define el *type hinting* o tipado

PEP 8 → Convenciones de nombres

Variables, parámetros, atributos y métodos:

Se nombran en *snake_case*, deben ser nombres descriptivos y sin prefijos que indiquen el tipo. **Ejemplo:**

```
1 class Factura():
2     def __init__(self, total_pagado, usuario_activo):
3         self.total_pagado = total_pagado
4         self.usuario_activo = usuario_activo
5     def calcular_total(self):
6         if self.usuario_activo:
7             return self.total_pagado * 0.9
8         else:
9             return self.total_pagado
10
11 total_pagado = 4
12 usuario_activo = True
13 factura = Factura(total_pagado, usuario_activo)
```

PEP 8 → Convenciones de nombres

Clases

Se nombran en *Camel/Case*, deben ser descriptivos y sin prefijos que indiquen el tipo.

Ejemplo

```
1 class Factura():
```

PEP 8 → Convenciones de nombres

Constantes

Se nombran en *UPPER_CASE*, deben ser descriptivos y sin prefijos que indiquen el tipo.

Ejemplo

```
1 MAX_INTENTOS = 3
```

PEP 8

→ Convenciones de nombres

Convención para atributos / métodos privados:

Permiten indicar que los mismos **no deben** accederse por fuera de la clase.

Se debe colocar un guión bajo como prefijo del nombre del atributo/método.

```
2  class Factura:
3      def __init__(self):
4          self.total = 0.0
5      def get_total(self) -> float:
6          return self.total
```



```
2  class Factura:
3      def __init__(self):
4          self._total = 0.0
5      def get_total(self) -> float:
6          return self._total
```

PEP 8 → Convenciones de nombres



EVITAR

Agregar prefijos que indiquen el tipo, o utilizar nombres que no sean descriptivos.

Ejemplo

```
7   oFactura = Factura()   # X
8   strNombre = "Juan"    # X
9   iContador = 0          # X
10  f = Factura()          # X
```

PEP 484

→ Tipado en funciones

Es posible definir tipos de datos para atributos o valores de retorno de métodos.

Ejemplos:

```
9  def aplicar_descuento(factura: Factura, porcentaje: int) -> float:
10      monto_con_descuento = factura.get_total() * (porcentaje / 100)
11      return monto_con_descuento
```

```
25  def crear_factura(cliente: Cliente, items: list[float]) -> float:
26      total = sum(items)
27      factura = Factura()
28      factura.set_total(total)
29      factura.asignar_cliente(cliente.get_nombre())
30      return total
```

Se suma esta norma a la PEP8 permitiendo **nombres claros + tipos claros**.

__str__

Es posible definir **métodos especiales** (también llamados *dunder methods*) que controlan cómo se comporta un objeto en distintas situaciones.

En el caso de `__str__`, define **cómo se representa un objeto como texto**.

Ejemplo:

```
1 class Factura:
2     def __init__(self, numero: int, total: float):
3         self.numero = numero
4         self.total = total
5
6     def __str__(self) -> str:
7         return f"Factura N° {self.numero} - Total: ${self.total}"
8
9
10 factura = Factura(1, 1500.0)
11 print(factura)
```


__str__

Es posible definir métodos especiales (también llamados *dunder methods*) que controlan cómo se comporta un objeto en distintas situaciones.

En el caso de `__str__`, define **cómo se representa un objeto como texto**.

Ejemplo:

Salida del print ⇒ `Factura N° 1 - Total: $1500.0`

Siguiente paso

Empezar a programar

